



수학 시뮬레이터 제작 보고서

수행평가

《시뮬레이션 파일 저장하는 방법》

- ① 최종 시뮬레이터의 html 코드를 전체 복사한다.
- ② 윈도우 메모장을 켜 후 위의 코드를 붙여넣기 한다.
- ③ 메모장에서 파일 - 다른 이름으로 저장 - 각각 본인이름1.html, 본인이름2.html으로 저장
(예 : 이현진1.html, 이현진2.html으로 꼭 확장자 붙여서 저장)

[첫 번째 시뮬레이터 제작 보고서]

1. 내가 선택한 첫 번째 문제 (캡처해서 붙여넣기 해도 됨)

문제 4 2024학년도 KAIST 기출

재확인 CHECK ☐ ☐ ☐

아래 그림과 같이 중심이 (0, 1)이고 반지름이 2인 원 X, 중심이 (1, 0)이고 반지름이 1인 원 Y, 중심이 (-1, 0)이고 반지름이 1인 원 Z가 있다. 점 A는 (0, 3)에서 시작하여 원 X를 따라 시계방향으로, 점 B는 (1, 1)에서 시작하여 원 Y를 따라 시계방향으로, 점 C는 (-1, 1)에서 시작하여 원 Z를 따라 반시계 방향으로 각각 일정한 속력으로 이동한다. 세 점 A, B, C가 동시에 출발하며 각 점이 원을 한 바퀴 도는 데 걸리는 시간이 같다.

- (1) 세 점 A, B, C가 각 원을 한 바퀴 도는 동안 한 직선 위에 있는 횟수를 구하라.
- (2) 세 점 A, B, C가 각 원을 한 바퀴 도는 동안 만드는 삼각형 ABC 넓이의 최댓값을 구하라.

2. 이 문제의 풀이 (반드시 손글씨로 작성하여 캡처하거나 패드에 손글씨로 작성한 것을 붙여넣으세요.)

$$\begin{aligned}
 (1) \quad & A = (2\sin\theta, 1+2\cos\theta) \\
 & B = (1+\sin\theta, \cos\theta) \\
 & C = (-1-\sin\theta, \cos\theta) \\
 & \Rightarrow \frac{\cos\theta+1}{\sin\theta-1} = 0 \quad \therefore \theta = \pi \text{ 때 한 직선} \\
 & \theta = \frac{3\pi}{2} \text{ 때 } B, C \text{ 한 직선 } A, B, C \text{ 한 직선} \\
 & \therefore 2\text{번} \\
 (2) \quad & \Delta ABC = (1+\sin\theta)(1+\cos\theta) = \sin\theta\cos\theta + \sin\theta + \cos\theta + 1 \\
 & \sin\theta + \cos\theta = a \\
 & \sin\theta\cos\theta = b \\
 & a^2 + b^2 = 1, -\sqrt{2} \leq a \leq \sqrt{2} \\
 & \therefore a + \frac{a^2-1}{2} + 1 = \frac{a^2}{2} + a + \frac{1}{2} \leq \frac{3}{2} + \sqrt{2}
 \end{aligned}$$

3. 이 문제가 시뮬레이션이 필요하다고 느낀 이유를 서술하세요.

이 문제는 세 점 A,B,C가 각각 다른 원 위를 동시에 움직이는 상황을 다루고 있기 때문에, 정지된 그림만으로는 전체 움직임을 이해하기 어렵다고 느꼈다. 특히 세 점의 출발 위치와 회전 방향이 서로 달라서, 시간이 지남에 따라 세 점이 어떤 위치 관계를 가지는지 머릿속으로만 추적하기에는 한계가 있었다.

가장 시뮬레이션이 필요하다고 느낀 부분은 세 점이 한 바퀴 도는 동안 항상 한 직선 위에 놓이는지 확인하는 과정이었다. 겉으로 보기에는 세 점이 서로 다른 원 위를 따로 움직이는 것처럼 보이지만, 실제로는 세 점의 y좌표가 같아져 같은 수평선 위에 놓인다. 슬라이더로 점들을 직접 움직여 보면 이 사실을 직관적으로 확인할 수 있다.

또한 이 문제는 삼각함수로 각 점의 좌표를 나타내야 하므로, 회전 방향이나 부호를 잘못 설정하기 쉽다. 시뮬레이터에서 현재 점의 위치와 좌표를 함께 표시하면 내가 세운 식이 실제 움직임과 일치하는지 검토할 수 있다.

마지막으로, 세 점이 움직이며 만들어내는 도형의 넓이를 구할 때도 시뮬레이션이 도움이 된다. 순간적으로는 세 점이 한 직선 위에 있어 삼각형 넓이가 0이지만, 시간이 지나면서 선분이 이동하며 일정한 영역을 쓸고 지나간다. 이 자취를 시각적으로 확인하면 넓이를 구하기 위한 풀이 방향을 잡는 데 도움이 될 것이라고 생각했다.

4. 사용한 AI 플랫폼을 모두 적으세요.

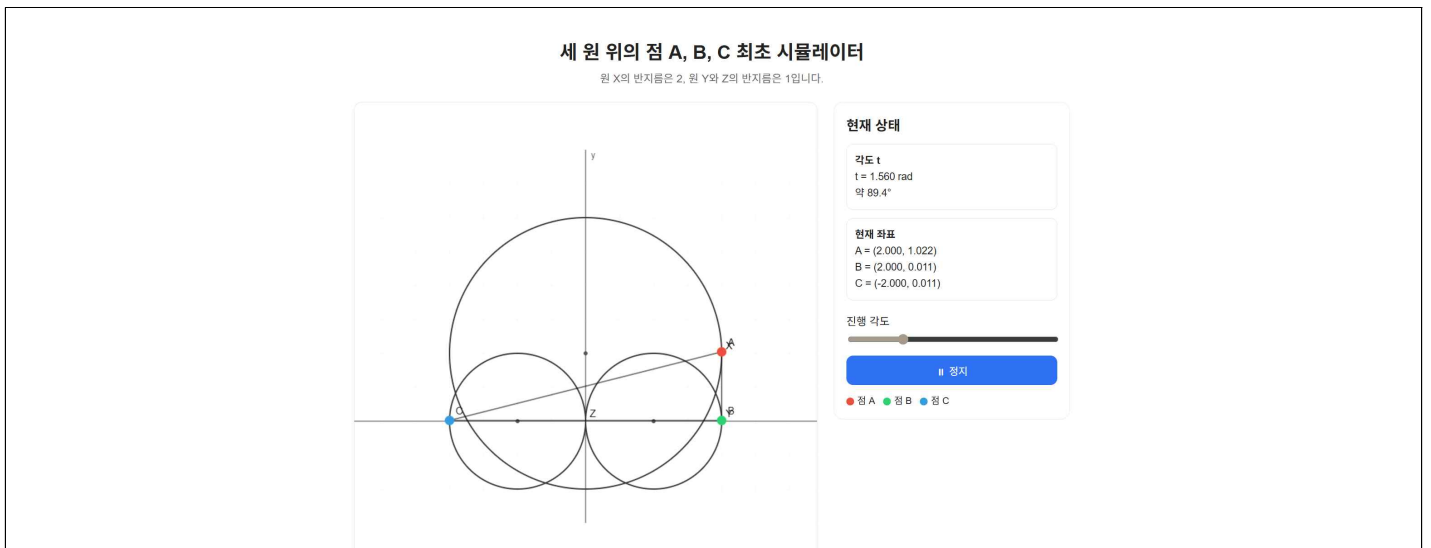
ChatGPT

5. 시뮬레이터를 만들기 위한 최초의 프롬프트를 작성하세요.

좌표평면 위에 중심이 (0,1)이고 반지름이 1인 원 X, 중심이 (1,0)이고 반지름이 1인 원 Y, 중심이 (-1,0)이고 반지름이 2인 원 Z를 그려줘. 점 A는 (0,3)에서 출발해서 원 X를 따라 시계방향으로 움직이고, 점 B는 (1,1)에서 출발해서 원 Y를 따라 시계방향으로 움직이고, 점 C는 (-1,1)에서 출발해서 원 Z를 따라 반시계방향으로 움직이게 해줘.

슬라이더를 움직이면 세 점 A,B,C가 동시에 움직이게 하고, 현재 세 점의 위치를 좌표평면에 표시해줘. 세 점을 선분으로 연결해서 현재 위치 관계도 볼 수 있게 해줘. 또한 자동 재생 버튼을 만들어서 세 점이 한 바퀴 도는 모습을 볼 수 있게 해줘.

6. 최초 시뮬레이터의 모습을 캡처해서 붙여넣으세요.



7. 최초 시뮬레이터에서 수정해야 할 사항을 서술하세요.

최초 시뮬레이터는 세 원과 세 점 A,B,C의 움직임을 시각적으로 보여주고, 슬라이더와 자동 재생 기능을 통해 문제 상황을 파악하는 데에는 도움이 되었다. 그러나 문제의 풀이에 직접적으로 활용하기에는 몇 가지 보완이 필요했다.

첫째, 세 점이 한 직선 위에 놓이는지에 대한 판정 기능이 부족했다. 문제의 핵심은 세 점 A,B,C가 원을 한 바퀴 도는 동안 같은 직선 위에 놓인다는 점인데, 최초 시뮬레이터에서는 단순히 세 점을 선분으로 연결해 보여줄 뿐, 왜 일직선이 되는지 설명해주지 않았다. 따라서 세 점의 y좌표를 비교하여 $y_A = y_B = y_C$ 가 되는지 확인하고, 현재 세 점이 놓인 직선의 방정식을 함께 표시하는 기능이 필요하다.

둘째, 좌표 표시가 소수점 근삿값으로만 나타나 수학적 풀이와 연결하기 어려웠다. 시뮬레이터에서는 점들의 위치가 (0.500,1.866)처럼 근삿값으로 표시되지만, 실제 풀이에서는 $\sin t$, $\cos t$ 와 같은 삼각함수식으로 좌표를 나타내야 한다. 따라서 현재 각도 t 에 따른 점 A,B,C의 좌표식을 함께 보여주면, 시뮬레이터가 단순한 그림이 아니라 풀이 검토 도구로 더 잘 활용될 수 있다.

셋째, 세 점이 움직이며 만들어내는 도형의 넓이를 이해하기 위한 자취 표시 기능이 부족했다. 문제에서는 세 점이 원을 한 바퀴 도는 동안 만들어지는 삼각형 또는 영역의 넓이를 묻고 있기 때문에, 현재 순간의 위치만 보여주는 것만으로는 전체 도형의 모양을 파악하기 어렵다. 따라서 점들이 움직이는 동안 선분이나 점의 자취를 누적해서 표시하고, 한 바퀴를 돌았을 때 어떤 영역이 만들어지는지 볼 수 있도록 수정할 필요가 있다.

마지막으로, 문제 풀이에 필요한 핵심 관찰을 안내하는 설명이 부족했다. 예를 들어 “세 점의 y좌표가 같기 때문에 항상 수평선 위에 놓인다”와 같은 설명이 추가되면, 사용자가 단순히 움직임을 관찰하는 데서 끝나지 않고 문제의 수학적 구조를 이해하는 데 도움을 받을 수 있다.

8. 수정하기 위한 프롬프트를 (2번째 ~ n번째) 모두 작성하세요.

2번째 프롬프트

처음 만든 시뮬레이터는 세 점 A,B,C가 움직이는 모습은 보여주지만, 현재 점들의 위치를 수학적으로 확인하기에는 부족하다. 오른쪽 정보창에 현재 각도 t 와 세 점 A,B,C의 좌표를 실시간으로 표시해줘.

이때 원 X의 반지름은 2이므로 점 A의 좌표는

$A=(2\sin t, 1+2\cos t)$ 이고, 점 B,C의 좌표는 각각 $B=(1+\sin t, \cos t)$, $C=(-1-\sin t, \cos t)$ 가 되도록 해줘.

좌표는 소수점 근삿값으로 표시하고, 각 점이 어떤 원 위에서 움직이는지도 함께 알 수 있게 정리해줘. 슬라이더를 움직일 때 좌표값이 바로 갱신되도록 만들어줘.

3번째 프롬프트

세 점 A,B,C가 한 직선 위에 놓이는 순간을 찾을 수 있는 기능을 추가해줘.

단, 자동 재생 중에는 일직선이 되는 순간이 너무 빠르게 지나가서 화면에서 바로 확인하기 어렵다. 따라서 화면에 단순히 “일직선 위에 있다/없다”를 계속 표시하는 방식은 사용하지 말고, 세 점이 일직선이 되는 순간을 감지하면 그때의 정보를 오른쪽 정보창에 기록해줘.

세 점의 좌표를 이용해 삼각형의 넓이가 거의 0이 되는 순간을 일직선이 되는 순간으로 판단해줘. 일직선이 되는 순간이 발견되면 오른쪽에 다음 내용을 목록으로 표시해줘.

현재 각도 t

그때의 점 A의 좌표

그때의 점 B의 좌표

그때의 점 C의 좌표또한 같은 일직선 순간이 너무 여러 번 중복 기록되지 않도록, 이미 기록된 t 값과 매우 가까운 경우에는 다시 기록하지 않게 해줘. 사용자가 자동 재생을 시켜도 일직선이 되는 순간들을 나중에 확인할 수 있도록 만드는 것이 목적이야.

4번째 프롬프트

세 점 A,B,C가 일직선이 되는 순간을 근사적으로 감지하지 말고, 좌표식을 이용해서 정확한 θ 값을 계산한 뒤 그 순간을 표시하도록 수정해줘.

5번째 프롬프트

좌표식으로 일직선을 구하는 과정을 시뮬레이터에서 빼고 일직선이 되는 점을 그 점을 한 번 지났을 때 표시해 그리고 다음에 또 지날 때는 표시하지 말고 그리고 초기화 버튼을 만들어서 그 버튼을 누르면 ABC가 초기 상태로 돌아가고 일직선으로 표시된 점을 빼는 걸 추가해줘

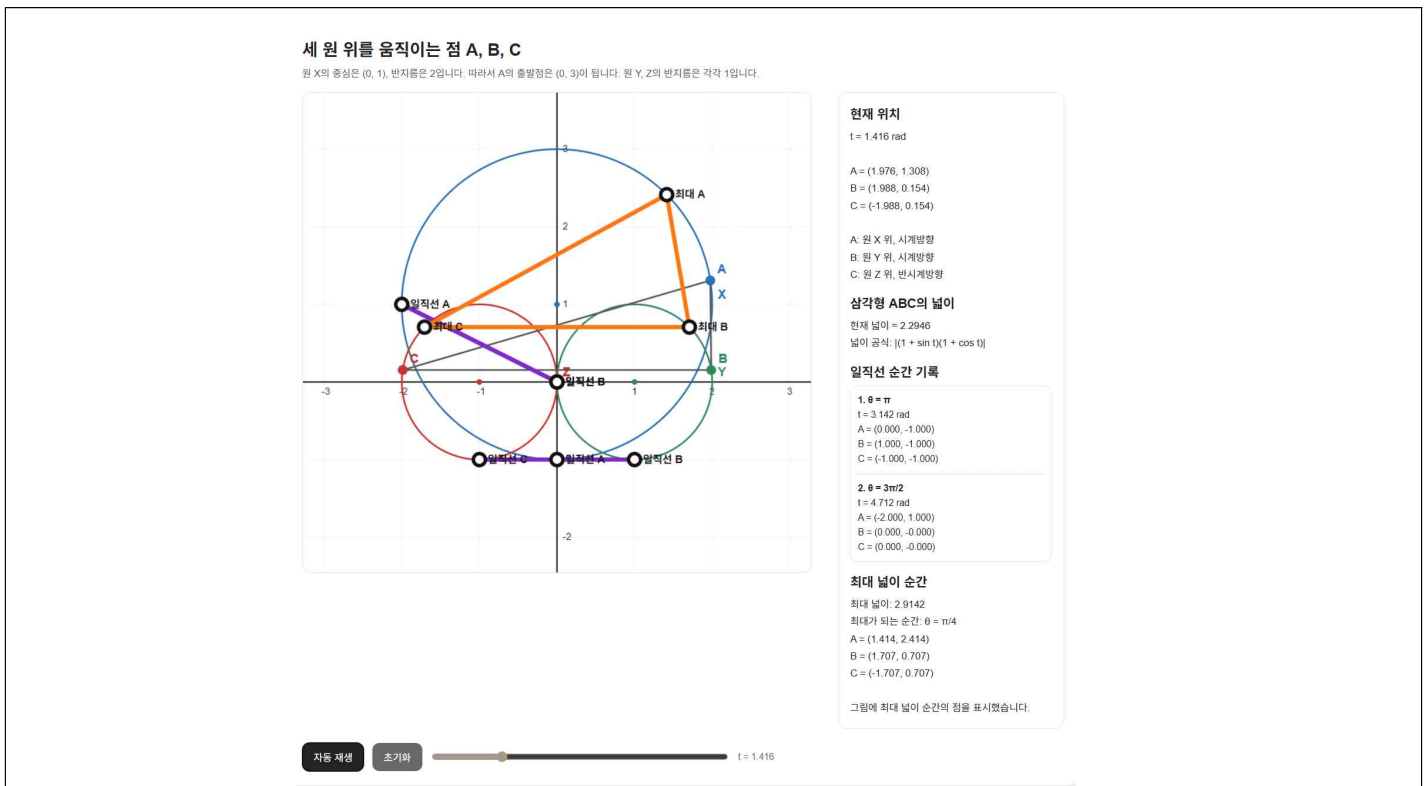
6번째 프롬프트

근데 b랑 c가 같아지는 이분의 3파이 지점에서는 표시가 안되고 있어 그것도 표시가 되게 해줘

7번째 프롬프트

그리고 abc가 이루는 삼각형의 넓이가 옆에 표시되게 해줘 그리고 넓이가 최대가 될때는 그때의 점들도 그림에 진하게 표시가 되게 해줘

9. 최종 시뮬레이터의 모습을 캡처해서 붙여넣으세요.



10. 자신이 만든 시뮬레이터를 처음 본 사람이 사용할 수 있도록 사용 방법을 서술하세요.

이 시뮬레이터는 세 점 A,B,C가 각각 정해진 원 위를 움직일 때, 세 점의 위치 변화와 삼각형 ABC의 넓이 변화를 관찰하기 위해 제작한 것이다. 처음 실행하면 좌표평면 위에 세 개의 원 X,Y,Z가 표시되고, 각 원 위에 점 A,B,C가 초기 위치에 놓여 있다.

사용자는 화면 아래쪽의 슬라이더를 움직여 각도 t 값을 직접 조절할 수 있다. 슬라이더를 움직이면 세 점이 동시에 이동하며, 현재 위치에서 세 점을 연결한 삼각형도 함께 표시된다. 오른쪽 정보창에서는 현재 각도 t 와 점 A,B,C의 좌표를 실시간으로 확인할 수 있다. 이때 점 A는 원 X 위에서 시계방향으로 움직이고, 점 B는 원 Y 위에서 시계방향, 점 C는 원 Z 위에서 반시계방향으로 움직인다.

화면 아래의 자동 재생 버튼을 누르면 세 점이 한 바퀴 도는 과정이 자동으로 재생된다. 다시 버튼을 누르면 재생이 멈춘다. 이를 통해 슬라이더를 직접 움직이지 않아도 세 점의 전체 운동 과정을 연속적으로 관찰할 수 있다. 오른쪽 정보창에는 삼각형 ABC의 현재 넓이도 함께 표시된다. 따라서 세 점의 위치가 변할 때 삼각형의 넓이가

어떻게 달라지는지 확인할 수 있다. 또한 세 점이 한 직선 위에 놓이는 순간이 지나가면, 그때의 t 값과 세 점의 좌표가 자동으로 기록된다. 기록된 일직선 순간은 좌표평면 위에서도 굵은 선으로 표시되어 나중에 다시 확인할 수 있다.

삼각형의 넓이가 최대가 되는 순간도 시뮬레이터에 표시된다. 최대 넓이가 되는 순간의 세 점은 현재 움직이는 점들과 구분되도록 다른 색의 굵은 선으로 연결되어 나타난다. 이를 통해 사용자는 세 점이 일직선이 되는 순간과 삼각형의 넓이가 최대가 되는 순간을 시각적으로 비교할 수 있다.

마지막으로 초기화 버튼을 누르면 점 A,B,C가 처음 위치로 돌아가고, 기록된 일직선 순간과 최대 넓이 표시가 모두 사라진다. 따라서 사용자는 처음 상태에서 다시 시뮬레이션을 실행하며 같은 과정을 반복해서 관찰할 수 있다.

11. 시뮬레이터가 자신이 의도한대로 문제를 파악하거나 풀이과정에 대한 힌트를 얻거나 풀이에 대한 검토를 하는 등에 도움이 되었는지, 도움이 되거나 되지 않는 부분에 대해서 구체적으로 서술하세요.

세 점이 일직선이 되는 순간을 자동으로 기록해 주는 기능은 풀이를 검토하는 데 도움이 되었다. 자동 재생을 실행하면 세 점이 계속 움직이기 때문에 일직선이 되는 순간이 눈으로 확인하기 전에 지나가 버릴 수 있다. 그런데 시뮬레이터에서는 그 순간을 지나면 해당 t 값과 점 A,B,C의 좌표가 오른쪽 정보창에 기록되므로, 나중에 멈춰서 다시 확인할 수 있었다. 이를 통해 내가 계산한 일직선 조건이 실제 움직임에서도 맞는지 비교할 수 있었다. 특히 $t = 3\pi/2$ 일 때는 B와 C가 같은 점에 놓이는데, 이 경우도 세 점이 한 직선 위에 있다고 볼 수 있다는 점을 시각적으로 확인할 수 있었다.

삼각형 ABC의 넓이가 실시간으로 표시되는 기능도 유용했다. 세 점이 움직일 때 삼각형의 모양만 보는 것이 아니라, 넓이 값이 어떻게 증가하고 감소하는지를 함께 확인할 수 있었기 때문이다. 또한 넓이가 최대가 되는 순간의 점들이 다른 색의 굵은 선으로 연결되어 표시되기 때문에, 현재 움직이는 삼각형과 최대 넓이일 때의 삼각형을 쉽게 비교할 수 있었다. 이 기능은 넓이 공식을 통해 구한 결과가 실제 도형의 변화와 일치하는지 검토하는 데 도움이 되었다.

하지만 시뮬레이터가 풀이 과정을 완전히 대신해 주는 것은 아니었다. 예를 들어 왜 특정 t 값에서 세 점이 일직선이 되는지, 왜 그 순간에 넓이가 최대가 되는지는 좌표식을 이용해 직접 계산해야 알 수 있다. 즉, 시뮬레이터는 정답을 증명해 주기보다는 문제 상황을 눈으로 이해하고, 계산 결과가 맞는지 확인하는 보조 도구에 가깝다.

따라서 이 시뮬레이터는 문제를 처음 파악하고 풀이 방향을 잡는 데 도움이 되었으며, 특히 일직선이 되는 순간과 넓이가 최대가 되는 순간을 시각적으로 비교하는 데 효과적이었다. 덕분에 좌표식으로 구한 결과가 실제 점들의 움직임과 잘 맞는지 확인할 수 있었고, 문제를 더 체계적으로 분석할 수 있었다.

[두 번째 시뮬레이터 제작 보고서]

1. 내가 선택한 두 번째 문제 (캡처해서 붙여넣기 해도 됨)

자연수 N 에 대하여 다음 조건을 만족시키는 모든 홀수 m 의 값의 합을 $f(N)$ 이라 하자.

(가) 등차수열 $\{a_n\}$ 은 첫째항이 자연수이고 공차가 1이다.

(나) $\sum_{k=1}^m a_k = N$ 이다.

예를 들어, $N = 21$ 인 경우에 아래의 두 가지만 가능하므로 $f(21) = 1 + 3 = 4$ 이다.

$$21 = \sum_{k=1}^1 (20 + k), \quad 21 = 6 + 7 + 8 = \sum_{k=1}^3 (5 + k)$$

자연수 N 과 홀수 m 에 대하여 다음 조건을 만족시키는 수열 $\{a_n\}$ 의 첫째항이 될 수 있는 자연수의 개수를 $g_N(m)$ 이라 할 때, 다음 물음에 답하시오.

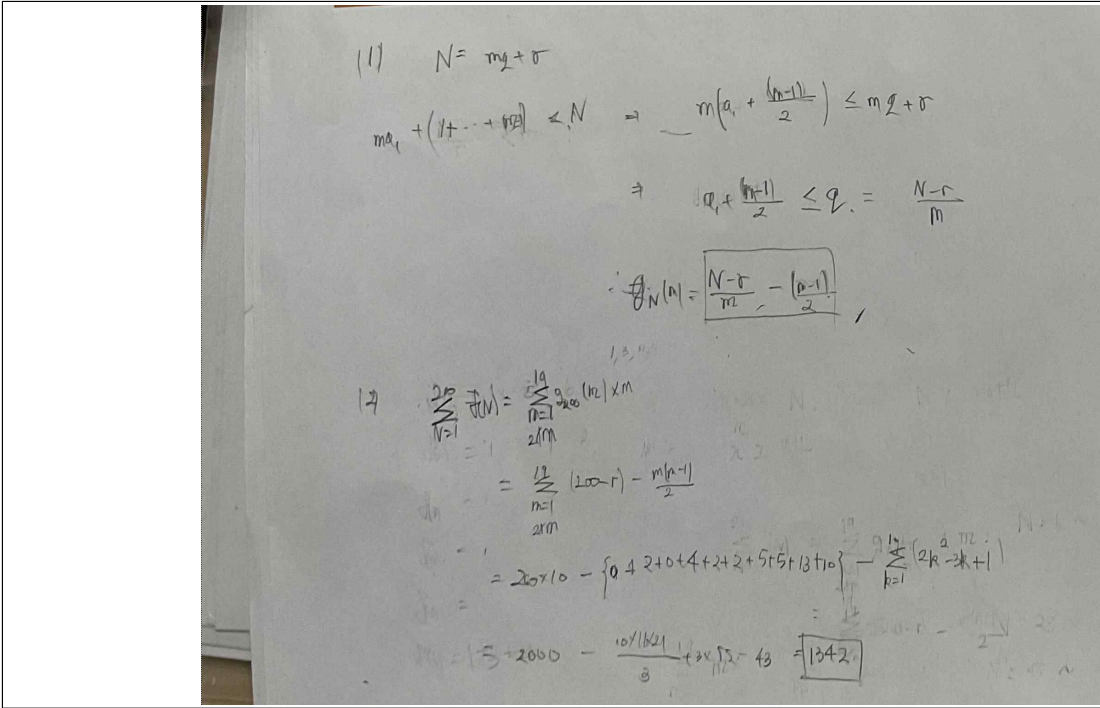
(가) 등차수열 $\{a_n\}$ 은 첫째항이 자연수이고 공차가 1이다.

(나) $\sum_{k=1}^m a_k \leq N$ 이다.

(1) N 을 m 으로 나눈 나머지가 r 일 때, $g_N(m)$ 을 N, m, r 를 이용하여 나타내시오.

(2) $\sum_{N=1}^{200} f(N)$ 의 값을 구하시오. 1342

2. 이 문제의 풀이 (반드시 손글씨로 작성하여 캡처하거나 패드에 손글씨로 작성한 것을 붙여넣으세요.)



3. 이 문제가 시뮬레이션이 필요하다고 느낀 이유를 서술하세요.

이 문제는 N , m , 첫째항 a_1 사이의 관계를 이용해 조건을 만족하는 경우의 수를 세는 문제이다. 식으로 정리하면

$$ma_1 + \frac{m(m-1)}{2} \leq N$$

이라는 조건이 나오지만, 처음에는 N 과 m 이 변할 때 가능한 첫째항의 개수가 어떻게 달라지는지 직관적으로 파악하기 어려웠다.

특히 $g_N(m)$ 은 고정된 N 과 m 에 대해 가능한 첫째항 a_1 의 개수를 의미하고, $f(N)$ 은 여러 m 에 대한 값을 모두 합한 것이므로 단순히 식만 보면 전체 구조가 잘 보이지 않는다. 따라서 N 과 m 을 직접 바꾸면서 $g_N(m)$ 의 변화와 가능한 a_1 의 범위를 확인할 수 있는 시뮬레이터가 필요하다고 느꼈다.

시뮬레이터를 사용하면 m 이 커질수록 기본 합 $\frac{m(m-1)}{2}$ 이 증가하여 가능한 첫째항의 개수가 줄어드는 모습을 표와 그래프로 확인할 수 있다. 이를 통해 문제의 조건을 더 쉽게 이해하고, 계산 결과가 맞는지 검토하는 데 도움을 받을 수 있다고 생각했다.

4. 사용한 AI 플랫폼을 모두 적으세요.

ChatGPT

5. 시뮬레이터를 만들기 위한 최초의 프롬프트를 작성하세요.

자연수 N 에 대해 공차가 1이고 첫째항이 자연수인 등차수열을 생각하자. 이때 앞에서부터 m 개 항의 합이 N 이하가 되는 경우를 확인할 수 있는 HTML 시뮬레이터를 만들어줘.

사용자가 N 값과 m 값을 슬라이더로 조절할 수 있게 해줘. 현재 선택된 N 과 m 에 대해, 앞에서부터 m 개 항의 합이 N 이하가 되도록 하는 첫째항 a_1 의 값을 자동으로 계산해서 보여줘.

등차수열의 앞에서부터 m 개 항의 합은 m 곱하기 a_1 에 m 곱하기 $m-1$ 을 2로 나눈 값을 더한 형태로 나타낼 수 있으므로, 오른쪽 정보창에 이 식과 가능한 첫째항의 목록, 그리고 가능한 첫째항의 개수 $g_N(m)$ 을 표시해줘. 또한 N 이 고정되어 있을 때 여러 m 에 대한 $g_N(m)$ 값을 표로 보여줘. 이를 통해 m 이 변할 때 가능한 첫째항의 개수가 어떻게 달라지는지 확인할 수 있게 해줘.

6. 최초 시뮬레이터의 모습을 캡처해서 붙여넣으세요.

등차수열 합 조건 시뮬레이터

자연수 N 에 대해 공차가 1이고 첫째항이 자연수인 등차수열에서, 앞에서부터 m 개 항의 합이 N 이하가 되는 첫째항 a_1 의 값을 확인하는 시뮬레이터입니다.

N 이 고정되었을 때 여러 m 에 대한 $g_N(m)$

m	가능한 첫째항 a_1	$g_N(m)$
1	1 ~ 21	21
2	1 ~ 10	10
3	1 ~ 6	6
4	1 ~ 3	3
5	1 ~ 2	2
6	1 ~ 1	1

현재 정보

$N = 21$

$m = 3$

현재 합 조건:

$$a_1 + a_2 + \dots + a_3 \leq 21$$

등차수열의 합으로 정리하면:

$$3a_1 + 3 \leq 21$$

가능한 첫째항의 개수:

$$g_{21}(3) = 6$$

등차수열: $a_n = a_1 + (n - 1)$

합 조건: $a_1 + a_2 + \dots + a_m \leq N$

정리한 식:

$$m \times a_1 + m(m - 1)/2 \leq N$$

가능한 첫째항 a_1

$a_1 = 1$

$a_1 = 2$

$a_1 = 3$

$a_1 = 4$

$a_1 = 5$

$a_1 = 6$

N 값  21

m 값  3

7. 최초 시뮬레이터에서 수정해야 할 사항을 서술하세요.

최초 시뮬레이터는 N 과 m 값을 바꾸면서 조건을 만족하는 첫째항 a_1 의 목록과 개수 $g_N(m)$ 을 확인하는 데에는 도움이 되었다. 또한 여러 m 에 대한 $g_N(m)$ 값을 표로 보여주어, m 이 커질수록 가능한 첫째항의 개수가 줄어드는 경향을 파악할 수 있었다.

하지만 문제 전체를 이해하기에는 표만으로는 변화가 한눈에 잘 보이지 않았다. 따라서 m 별 $g_N(m)$ 값을 막대그래프로 나타내어, 어떤 m 에서 경우의 수가 많고 적은지 시각적으로 비교할 수 있도록 수정할 필요가 있었다.

또한 문제에서는 각각의 $g_N(m)$ 뿐만 아니라, 모든 m 에 대한 값을 합친 $f(N)$ 도 중요하다. 최초 시뮬레이터에는 $f(N)$ 을 자동으로 계산해 주는 기능이 없었기 때문에, N 이 주어졌을 때 전체 경우의 수를 바로 확인하기 어려웠다. 따라서 $f(N)$ 값을 오른쪽 정보창에 함께 표시하는 기능이 필요했다.

마지막으로 N 값을 하나씩 직접 움직이는 방식만으로는 N 이 변할 때 전체적인 변화 양상을 관찰하기 불편했다. 그

래서 N 이 자동으로 증가하면서 표와 그래프가 함께 바뀌는 기능을 추가하면, N 에 따른 $g_N(m)$ 과 $f(N)$ 의 변화를 더 쉽게 파악할 수 있을 것이라고 생각했다.

8. 수정하기 위한 프롬프트를 (2번째 ~ n번째) 모두 작성하세요.

2번째 프롬프트

현재 시뮬레이터는 N 과 m 을 조절하면 조건을 만족하는 첫째항 a_1 의 목록과 $g_N(m)$ 값을 표로 보여준다. 하지만 표만으로는 m 이 변할 때 $g_N(m)$ 이 어떻게 달라지는지 한눈에 파악하기 어렵다.

따라서 고정된 N 에 대해 여러 m 의 $g_N(m)$ 값을 막대그래프로 표시해줘. 가로축은 m , 세로축은 $g_N(m)$ 이 되도록 하고, 현재 선택된 m 에 해당하는 막대는 다른 색으로 강조해줘.

3번째 프롬프트

문제에서 $f(N)$ 은 홀수 m 에 대한 $g_N(m)$ 값을 모두 더한 값이므로, 현재 N 에 대한 $f(N)$ 값을 자동으로 계산해서 오른쪽 정보창에 표시해줘.

가능한 모든 m 에 대해 $g_N(m)$ 을 구한 뒤 그 합을 $f(N)$ 으로 나타내줘. N 슬라이더를 움직이면 $f(N)$ 값도 바로 갱신되도록 해줘.

4번째 프롬프트

N 값이 변할 때 $g_N(m)$ 과 $f(N)$ 이 어떻게 달라지는지 관찰할 수 있도록 N 자동 증가 버튼을 추가해줘.

버튼을 누르면 N 값이 1부터 200까지 자동으로 증가하고, 그때마다 표, 막대그래프, 오른쪽 정보창이 실시간으로 갱신되게 해줘. 버튼을 다시 누르면 자동 증가가 멈추도록 하고, 버튼 글자도 “ N 자동 증가”와 “정지”로 바뀌게 해줘.

5번째 프롬프트

초기화 버튼을 추가해줘. 초기화 버튼을 누르면 N 은 21, m 은 3으로 돌아가고, 자동 증가도 멈추게 해줘. 표, 막대 그래프, 오른쪽 정보창도 처음 상태로 다시 표시되게 해줘.

6번째 프롬프트

오른쪽 정보창과 표를 더 보기 좋게 정리해줘. 오른쪽에는 현재 N , m , 합 조건식, 가능한 첫째항 a_1 , $g_N(m)$, $f(N)$ 을 차례대로 표시해줘.

표에는 각 m 에 대해 가능한 첫째항의 범위와 $g_N(m)$ 값을 표시해줘. 처음 보는 사람도 시뮬레이터의 의미를 이해할 수 있도록 제목과 설명도 간단히 추가해줘

등차수열 합 조건 시뮬레이터

공차가 1이고 첫째항이 자연수인 등차수열에서, 앞에서부터 홀수 개 m 개 항의 합이 N 이하가 되는 경우를 확인하는 시뮬레이터입니다. m 은 항상 홀수만 선택되며, $f(N)$ 은 각 홀수 m 에 대해 $m \times g_N(m)$ 을 더한 값으로 계산합니다.

홀수 m 별 $m \times g_N(m)$ 막대그래프

m	$m \times g_N(m)$
1	24
3	21
5	10

N 이 고정되었을 때 홀수 m 에 대한 값

홀수 m	가능한 첫째항 a_1	$g_N(m)$	$m \times g_N(m)$
1	1 ~ 24	24	24
3	1 ~ 7	7	21
5	1 ~ 2	2	10

현재 정보

$N = 24$
 $m = 3$ (홀수)

합 조건식:
 $3a_1 + 3 \leq 24$

가능한 첫째항 a_1 :
1, 2, 3, 4, 5, 6, 7

$g_N(m) = 7$
현재 값으로 쓰면: $g_{24}(3) = 7$

$m \times g_N(m) = 3 \times 7 = 21$

$f(N) = 55$
현재 값으로 쓰면: $f(24) = 55$

합 조건식

등차수열: $a_n = a_1 + (n - 1)$
합 조건: $a_1 + a_2 + \dots + a_m \leq N$
정리한 식:
 $m \times a_1 + m(m - 1)/2 \leq N$
단, m 은 홀수만 사용

가능한 첫째항 a_1

$a_1 = 1$ $a_1 = 2$ $a_1 = 3$ $a_1 = 4$
 $a_1 = 5$ $a_1 = 6$ $a_1 = 7$

$f(N)$ 계산 과정

$m = 1$: $g_{24}(1) = 24$, $1 \times 24 = 24$
 $m = 3$: $g_{24}(3) = 7$, $3 \times 7 = 21$
 $m = 5$: $g_{24}(5) = 2$, $5 \times 2 = 10$

따라서
 $f(24) = 24 + 21 + 10$
 $f(24) = 55$

N 값:

m 값:

1. N 값 조절
 조작 방법: 화면 아래쪽의 N 값 슬라이더를 움직인다.
 기능: 자연수 N의 값이 바뀌면서, 조건을 만족하는 홀수 m과 첫째항 a1의 개수가 어떻게 달라지는지 확인할 수 있다. N 값이 바뀌면 표, 막대그래프, 오른쪽 정보창이 함께 갱신된다.
2. m 값 조절
 조작 방법: 화면 아래쪽의 m 값 슬라이더를 움직인다.
 기능: m은 항상 홀수만 선택된다. 선택한 m에 대해 앞에서부터 m개 항의 합이 N 이하가 되는 첫째항 a1의 목록과 개수 $g_N(m)$ 을 확인할 수 있다.

3. 가능한 첫째항 a_1 확인

조작 방법: 오른쪽 정보창의 ‘가능한 첫째항 a_1 ’ 부분을 확인한다.

기능: 현재 N 과 m 에 대해 조건을 만족하는 첫째항 a_1 의 값들이 표시된다. 이를 통해 어떤 첫째항을 선택했을 때 m 개 항의 합이 N 이하가 되는지 알 수 있다.

4. $g_N(m)$ 과 $f(N)$ 계산 과정 확인

조작 방법: 오른쪽 정보창의 현재 정보와 $f(N)$ 계산 과정 부분을 확인한다.

기능: 선택한 m 에 대한 $g_N(m)$ 값이 표시되고, 각 홀수 m 에 대해 $m \times g_N(m)$ 을 계산한 뒤 이를 모두 더해 $f(N)$ 을 구하는 과정을 볼 수 있다.

5. 막대그래프와 표 확인

조작 방법: 화면 왼쪽의 막대그래프와 표를 확인한다.

기능: 막대그래프는 홀수 m 별 $m \times g_N(m)$ 의 값을 보여준다. 현재 선택된 m 은 다른 색으로 강조되어, 선택한 m 이 전체 $f(N)$ 계산에서 어느 정도 영향을 주는지 쉽게 비교할 수 있다. 표에서는 각 홀수 m 에 대한 가능한 첫째항 범위, $g_N(m)$, $m \times g_N(m)$ 을 확인할 수 있다.

6. N 자동 증가 및 초기화

조작 방법: ‘ N 자동 증가’ 버튼을 누르면 N 값이 자동으로 증가한다. 다시 누르면 멈춘다. ‘초기화’ 버튼을 누르면 N 은 21, m 은 3으로 돌아간다.

기능: N 이 변할 때 $g_N(m)$, $m \times g_N(m)$, $f(N)$ 이 어떻게 달라지는지 연속적으로 관찰할 수 있다. 초기화 버튼을 이용하면 처음 상태에서 다시 확인할 수 있다.

11. 시뮬레이터가 자신이 의도한대로 문제를 파악하거나 풀이과정에 대한 힌트를 얻거나 풀이에 대한 검토를 하는 등에 도움이 되었는지, 도움이 되거나 되지 않는 부분에 대해서 구체적으로 서술하세요.

이 시뮬레이터는 문제의 조건을 이해하고 풀이 과정을 검토하는 데 도움이 되었다. 처음에는 N , 홀수 m , 첫째항 a_1 사이의 관계가 식으로만 주어져 있어서 경우의 수가 어떻게 계산되는지 한눈에 파악하기 어려웠다. 하지만 시뮬레이터에서 N 과 m 값을 직접 바꾸어 보면서 가능한 첫째항 a_1 의 목록과 $g_N(m)$ 값을 바로 확인할 수 있어 문제 구조를 더 쉽게 이해할 수 있었다.

특히 m 이 홀수만 가능하다는 조건을 반영한 점이 유용했다. 처음에는 $f(N)$ 을 단순히 $g_N(m)$ 값들의 합으로 생각하기 쉬웠지만, 시뮬레이터에서 각 홀수 m 에 대해 m 곱하기 $g_N(m)$ 을 계산하고 이를 모두 더하는 과정을 확인하면서 $f(N)$ 의 의미를 더 정확히 이해할 수 있었다. 즉, 조건을 만족하는 경우의 개수만 세는 것이 아니라, 그때의 홀수 m 값 자체를 모두 더해야 한다는 점을 검토할 수 있었다.

또한 막대그래프와 표를 통해 어떤 m 이 $f(N)$ 값에 크게 영향을 주는지도 비교할 수 있었다. 표에서는 각 홀수 m 에 대해 가능한 첫째항의 범위, $g_N(m)$, m 곱하기 $g_N(m)$ 이 함께 표시되어 계산 과정을 확인하기 쉬웠다. 이를 통해 내가 손으로 계산한 결과가 맞는지 비교하는 데 도움이 되었다.

다만 시뮬레이터가 풀이를 완전히 대신해 주지는 않았다. 왜 합 조건이 m 곱하기 a_1 에 $m(m-1)/2$ 를 더한 형태로 정리되는지, 왜 $f(N)$ 을 m 곱하기 $g_N(m)$ 의 합으로 계산해야 하는지는 직접 수학적으로 설명해야 했다. 따라서 이 시뮬레이터는 문제 이해와 계산 검토에는 도움이 되었지만, 최종적인 식 유도과 논리적 서술은 학생이 직접 해야 한다고 느꼈다.

[마무리]

1. 두 문제에 대한 시뮬레이터를 기획하고 활용하여 문제를 해결하는 전 과정에서, 생성형 AI가 제공할 수 있는 가치와 반드시 인간(학생 본인)이 주도해야 하는 수학적 사고의 영역을 구분하여 서술하세요.

두 문제에 대한 시뮬레이터를 기획하고 활용하는 과정에서 생성형 AI는 복잡한 수학적 상황을 시각화하는 데 큰 도움을 주었다. 첫 번째 문제에서는 세 점 A,B,C가 각각 다른 원 위에서 움직일 때 세 점의 위치 관계, 일직선이 되는 순간, 삼각형의 넓이 변화를 시뮬레이터로 확인할 수 있었다. 두 번째 문제에서는 원 위의 점 P가 움직일 때 정사각형 OPQR이 어떻게 변하고, 선분 QR이 지나가는 전체 범위가 어떤 모양으로 나타나는지 시각적으로 볼 수 있었다.

이 과정에서 생성형 AI가 제공한 가장 큰 가치는 아이디어를 실제 작동하는 시뮬레이터로 빠르게 구현해 준 것이었다. 내가 문제 조건과 원하는 기능을 프롬프트로 제시하면, AI는 이를 HTML 코드로 바꾸어 슬라이더, 자동 재생, 좌표 표시, 자취 표시 같은 기능을 만들어 주었다. 또한 처음 만든 시뮬레이터의 부족한 점을 바탕으로 추가 프롬프트를 작성하면, QR의 전체 통과 범위 색칠이나 점 Q,R의 자취 표시처럼 풀이에 필요한 시각적 요소를 보완할 수 있었다. 즉, AI는 문제 상황을 눈으로 확인할 수 있는 도구를 만드는 데 효과적이었다.

하지만 수학적 사고의 핵심은 인간이 주도해야 한다고 느꼈다. 예를 들어 어떤 값을 슬라이더로 조절해야 하는지, 어떤 좌표식을 사용해야 하는지, 어떤 부분이 문제 풀이에 중요한지 판단하는 것은 내가 해야 했다. 첫 번째 문제에서 세 점의 좌표식을 세우고, 일직선 조건이나 넓이 공식을 생각하는 과정은 단순히 시뮬레이터가 대신해 줄 수 없었다. 두 번째 문제에서도 $P=(a+a\cos\theta, a\sin\theta)$ 로 두고, 정사각형의 회전 관계를 이용해 Q,R의 좌표를 해석하는 것은 학생 본인이 이해해야 하는 부분이었다.

또한 시뮬레이터는 결과를 시각적으로 보여줄 수는 있지만, 그것이 왜 성립하는지 엄밀하게 증명해 주지는 않는다. 예를 들어 QR이 지나가는 범위가 특정한 모양으로 보인다고 해서 그것이 곧 수학적 증명이 되는 것은 아니다. 실제 보고서나 풀이에서는 좌표식, 대수적 계산, 기하적 관계를 이용해 논리적으로 설명해야 한다. 따라서 생성형 AI는 문제를 파악하고 계산 결과를 검토하는 보조 도구로 사용할 수 있지만, 최종적인 풀이 방향 설정과 증명 과정은 인간이 책임지고 수행해야 한다.

결론적으로 생성형 AI는 시뮬레이터 제작, 시각화, 코드 구현, 반복 수정 과정에서 매우 유용한 도구였다. 그러나 문제의 핵심을 파악하고, 어떤 수학적 개념을 적용할지 결정하며, 결과를 논리적으로 해석하고 증명하는 과정은 반드시 학생 본인이 주도해야 한다고 생각한다. AI는 수학적 사고를 대신하는 존재가 아니라, 내가 세운 수학적 아이디어를 더 잘 관찰하고 검토할 수 있게 도와주는 협력 도구라고 볼 수 있다.

2. 복잡한 수학 문제를 AI와 협업하여 시뮬레이터로 구현해 본 경험이, 향후 자신이 희망하는 진로 분야에서 실무적인 문제를 해결하는 데 어떤 시사점을 주는지 서술하세요.

이번 활동을 통해 복잡한 문제를 해결할 때, 먼저 상황을 수학적으로 모델링하고 이를 시뮬레이션으로 확인하는 과정이 중요하다는 것을 느꼈다. 처음에는 식과 그림만으로 문제를 이해하려 했지만, 직접 시뮬레이터로 구현하자 점과 선분의 움직임, 그리고 전체 영역의 변화가 훨씬 명확하게 보였다. 이는 실제 문제를 해결할 때도 단순히 이론적으로 계산하는 것보다, 모델을 만들고 결과를 눈으로 확인하는 과정이 필요하다는 점을 보여준다.

이러한 경험은 내가 관심을 가지고 있는 로봇공학과도 연결된다. 로봇을 설계하거나 제어할 때도 실제 장치를 바로 제작하기보다, 먼저 움직임을 수학적으로 표현하고 시뮬레이션으로 검토하는 과정이 필요하다. 예를 들어 로봇의 관절이 목표 위치까지 도달할 수 있는지, 이동 과정에서 구조물과 충돌하지 않는지, 정해진 경로를 안정적으로 따

라갈 수 있는지와 같은 실무적인 문제는 제작 후에 확인하면 시간과 비용이 많이 든다. 따라서 사전에 시뮬레이션을 통해 문제를 예측하고 수정하는 과정이 중요하다.

AI는 이 과정에서 아이디어를 빠르게 시뮬레이터 형태로 구현하는 데 도움을 줄 수 있다. 이번 활동에서도 내가 원하는 기능을 설명하면 AI가 코드를 작성해 주었고, 이후 필요한 기능을 추가하며 시뮬레이터를 점점 개선할 수 있었다. 그러나 어떤 변수를 설정할지, 결과가 실제 상황에 맞는지, 시뮬레이션이 물리적으로 타당한지는 결국 사람이 판단해야 한다.

따라서 이번 경험은 로봇공학에서 AI를 단순히 코드를 대신 작성하는 도구가 아니라, 실무적인 문제를 미리 예측하고 설계를 검토하는 협업 도구로 활용할 수 있다는 점을 보여주었다. 동시에 AI가 만든 결과를 그대로 받아들이기 보다, 수학적 원리와 실제 조건을 바탕으로 직접 확인하고 수정하는 태도가 중요하다고 느꼈다.

3. 보고서를 작성하며 느낀점을 간단히 작성하세요.

이번 보고서를 작성하면서 수학 문제를 단순히 계산으로만 해결하는 것이 아니라, 문제 상황을 시각화하고 직접 조작해 보는 과정이 중요하다는 것을 느꼈다. 처음에는 점이나 선분의 움직임을 머릿속으로만 이해하려고 해서 전체적인 변화가 잘 보이지 않았지만, 시뮬레이터를 만들고 활용하면서 도형의 움직임과 자취를 훨씬 쉽게 파악할 수 있었다.

또한 AI를 활용하면 내가 생각한 아이디어를 빠르게 시뮬레이터로 구현할 수 있다는 점이 인상적이었다. 하지만 시뮬레이터가 보여주는 결과를 그대로 받아들이는 것이 아니라, 왜 그런 결과가 나오는지 수학적으로 해석하고 검토하는 과정은 직접 해야 한다는 것도 알게 되었다.

이번 활동을 통해 AI는 문제 해결을 도와주는 유용한 도구이지만, 최종적인 수학적 판단과 논리적인 설명은 학생 스스로 해야 한다는 점을 깨달았다. 앞으로도 복잡한 문제를 해결할 때 시뮬레이션과 수학적 사고를 함께 활용하고 싶다.